DAILY DATA DEEP DIVES · THE LATEST NEWS, FULLY EXPLAINED

Storage Wars

Snowflake moves CDC inside Postgres, and AI reviewers reshape Postgres 19

Issue #13 | Sat, 17 Oct 2026 | ⏱️ ~35 min read | Level 3/5

🗄️ Today's two stories

1. **Snowflake's Postgres-to-Snowflake data mirroring is now generally available.** Instead of an external tool reading the Postgres change stream, a Postgres extension writes every committed change into Iceberg tables on object storage, and a serverless task in Snowflake applies them. We explain how it works, why it applies changes as deletes plus appends instead of `MERGE`, and the gotchas, including one that can make your application's `UPDATE`s fail.
2. **PostgreSQL 19 will ship with roughly half the major features one of its committers expected, and AI tools are part of the reason.** Peter Eisentraut's post on the wave of reverts, LWN's write-up and The Register's coverage this week explain what happened: AI-assisted bug hunting found defects faster than the project could fix them, and a record run of security fixes ate the beta period. We go through the numbers, what's in and out of 19, and why you should expect more Postgres patch releases from 2027.
3. **On the radar:** 7 more things that happened, listed at the end.

STORY 1 Snowflake moves CDC inside Postgres

SNOWFLAKE POSTGRES · DATA MIRRORING GA 15 OCT 2026 · AWS AND AZURE · ~17 MIN READ

Almost every team that runs Postgres for its app and a warehouse for its analytics has built the same plumbing: a tool reads the Postgres change stream, pushes the changes somewhere, and something else merges them into warehouse tables. Each piece can fall behind, lose its place or disagree with the others about what happened. Snowflake's answer, now generally available, is to move the hardest part of that pipeline into Postgres itself. The database writes its own changes, in transaction-sized batches, straight into open-format tables on object storage. It's an unusual design, and it's worth understanding even if you never use Snowflake.

WHAT HAPPENED

On **15 October 2026**, Snowflake marked **data mirroring for Snowflake Postgres** as generally available.[1] Mirroring continuously copies tables from a Snowflake Postgres instance into a Snowflake database, keeps up with schema changes, and lands the data either as ordinary Snowflake tables or as Snowflake-managed Iceberg tables.[2] It's available on AWS and Azure, in every region where Snowflake Postgres is offered, and a mirror can't cross accounts, regions or cloud providers.[2]

The feature has been public for four months. Snowflake announced it on **16 June 2026**, saying mirroring would reach public preview "soon" and that mirroring in the other direction, from Snowflake to Postgres, was planned for later in the year.[3] By **23 July**, when Snowflake engineer Marco Slot published a detailed design post, it was in public preview.[4]

Snowflake's own description of what you get is short. There's no external CDC service or connector to run. You set it up with one SQL call or a few clicks. New tables and schema changes in mirrored schemas are picked up automatically. Each mirrored table gets a `$live` view that shows committed changes within seconds, and a `$changes` table holding seven days of row-level history that you can query from both Snowflake and Postgres.[3,2]

Two customer claims came with the June announcement. Ericsson said it consolidated four legacy databases onto Snowflake Postgres and cut data lag on its customer support platform from 48 hours to under an hour. SimCorp said it cut synchronization time for one service from several hours to under 20 minutes.[3] Those are customer statements in a vendor post, not measurements you can check, and they describe the platform as a whole rather than mirroring alone. Snowflake hasn't published throughput or latency benchmarks for mirroring itself.[5,4]

THE BACKGROUND YOU NEED

Snowflake Postgres, and where it came from

Snowflake bought the Postgres company Crunchy Data in June 2025, for a reported $250 million.[6] On **24 February 2026**, **Snowflake Postgres** became generally available: managed Postgres instances that you create from Snowflake, each running on a dedicated virtual machine, reachable with any Postgres client, in selected AWS and Azure regions.[7]

The piece that makes mirroring possible is **pg_lake**, a set of extensions that came with Crunchy and that Snowflake open-sourced under the Apache 2.0 licence in November 2025.[6,8] pg_lake lets Postgres read and write **Apache Iceberg** *≈ an open table format: a table stored as Parquet data files on object storage, plus metadata files that track which files make up each version (snapshot) of the table* tables directly. Its README describes two halves: Postgres extensions (`pg_lake_iceberg`, which "implements the Iceberg specification", and `pg_lake_table`, a **foreign data wrapper** *≈ a Postgres extension type that makes data outside the database look like a table you can query with SQL* for files in object storage), and **pgduck_server**, a separate multi-threaded process that runs **DuckDB** *≈ an embedded, column-oriented analytical database engine, built for fast scans and aggregations* and speaks the Postgres wire protocol. Postgres plans the query and handles transactions, then "delegates scanning of the data and the computation" to DuckDB. Keeping DuckDB in its own process "avoids the threading and memory-safety limitations" of putting a multi-threaded engine inside Postgres, which is built around process isolation rather than threads.[8]

How Postgres change data capture normally works

Change data capture *≈ CDC: capturing every insert, update and delete in a database as a stream of events, so another system can stay in sync* on Postgres starts with the **write-ahead log** *≈ WAL: the append-only log where Postgres records every*

change before applying it to table files, used for crash recovery and replication. With `wal_level = logical`, Postgres can run **logical decoding** *≈ turning WAL records back into row-level changes (this row was inserted, that row was updated) for consumers outside the server*, which an **output plugin** *≈ code that formats decoded changes for a consumer, such as the built-in pgoutput used by logical replication* turns into a stream.[9]

Consumers read that stream through a **replication slot** *≈ a named, crash-safe bookmark on the server that remembers how far one consumer has read, and keeps the WAL it still needs*. The docs are blunt about the trade-off. Slots "persist across crashes and know nothing about the state of their consumer(s)", and they keep WAL and old catalog rows from being removed even when no one is connected. "In extreme cases this could cause the database to shut down to prevent transaction ID wraparound."[9] They also warn that a slot's position is saved only at checkpoints, so after a crash a consumer can see recent changes again, and "logical decoding clients are responsible for avoiding ill effects from handling the same message more than once."[9]

For updates and deletes, the stream has to say which row changed. That's the table's **replica identity** *≈ the setting that decides what Postgres writes to the WAL to identify the old version of an updated or deleted row: the primary key by default, a chosen unique index, every column (FULL), or nothing*. A table that publishes updates but has a replica identity of `NOTHING` can't be updated at all: Postgres rejects the statement with "cannot update table … because it does not have a replica identity and publishes updates", and the same for deletes. Inserts still work.[10] Hold on to that one; it matters later.

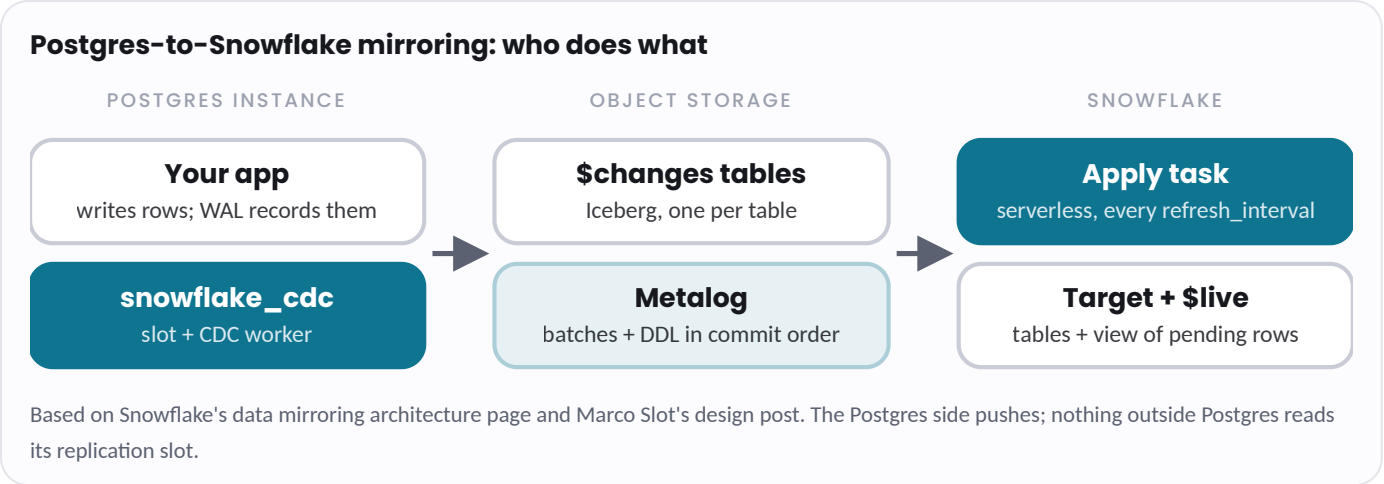
So a typical pipeline looks like this: a **Debezium** *≈ a widely used open-source CDC tool that reads database change logs and publishes the changes, usually to Kafka*-style connector reads the slot, writes changes to a queue or files, and a job in the warehouse runs an **upsert** *≈ an insert-or-update: if a row with this key exists, update it, otherwise insert it*, usually as a SQL `MERGE`, into the target tables. Every arrow in that picture is a separate system with its own failure modes.

HOW IT WORKS

The pieces

Mirroring splits the pipeline into a Postgres half and a Snowflake half, joined by Iceberg tables on object storage. The architecture page lists the parts:[2]

- On Postgres: a **publication** *≈ a Postgres object that names the set of tables whose changes are published to logical replication consumers* created by a new extension, `snowflake_cdc`, and a logical decoding slot read by a **CDC background worker** inside Postgres.
- In the middle: for each source table, an Iceberg **change log table** with the suffix `$changes`, written by the worker through pg_lake, and one Iceberg **metalog** table per mirror that records schema changes and change batches "in commit order".
- On Snowflake: the target database, the per-table target tables, `$live` views, and a scheduled **serverless task** that calls an apply procedure.



Pushing instead of pulling

The interesting decision is which side drives. In most CDC designs an outside process pulls from the slot. Here the worker lives inside Postgres and pushes. Slot's post explains why: with plain logical decoding, the consumer is blind to what Postgres is doing, such as schema changes, snapshot alignment, or whether Postgres or the network has failed. An extension "knows exactly what is happening in Postgres."[4] He also argues that object storage is the right place to put changes, because it's scalable and reliable and Postgres already uses it for backups.[4]

Every write passes through four stages: the row change and its WAL entry; **decode**, where WAL becomes row-level changes; **capture**, where changes are grouped into batches; and **apply**, where batches are merged into the destination.[4] The decoder reads catalog tables as they were when the WAL was written, a "historic snapshot", so a record stays readable even if the table was altered or dropped since. Decoded changes go to temporary files. When the decoder is told to finish a batch, capture appends those files to the Iceberg change logs, writes a metalog record and tracks the **LSN** ≈ *log sequence number: a position in the WAL, used to say exactly how far a consumer has read* it has replicated up to.[4] The performance page puts the cadence at roughly one batch "about every 15 seconds".[5]

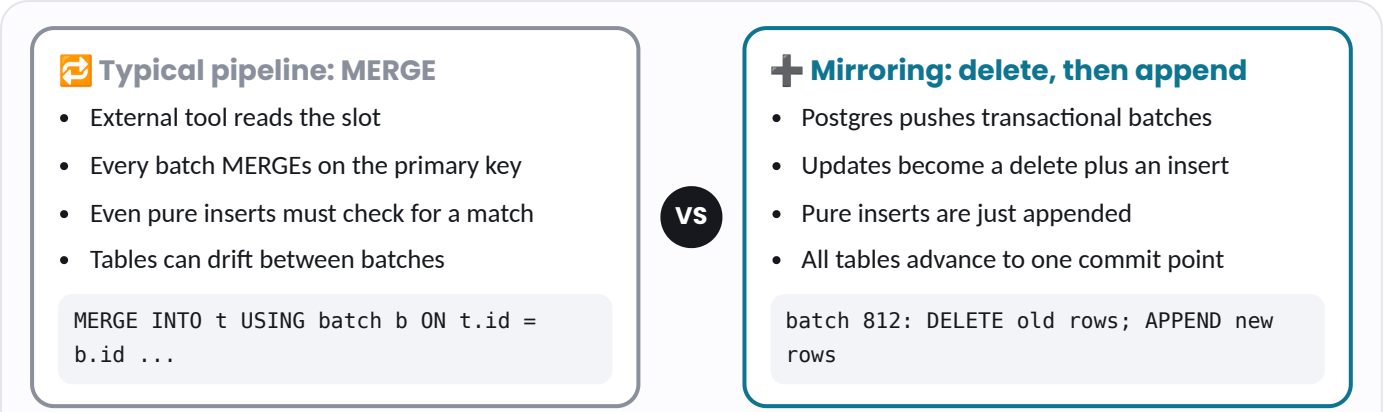
The key property is in how those writes are grouped. Postgres pushes a batch, across several tables' change logs plus any schema change, in **one Postgres-side transaction**. Snowflake applies one or more batches in **one Snowflake-side transaction**. So all target tables move forward "exactly to a Postgres transaction boundary".[4] If an order and its line items were committed together in Postgres, you never see one without the other in Snowflake, and joins across mirrored tables stay correct.

If an apply run fails, it rolls back, and the next run resumes from the same metalog position. Snowflake's docs present that as exactly-once delivery.[2] Slot's post adds that if the WAL a mirror needs is ever lost, Postgres can push fresh snapshots and tell Snowflake to consume them, which he calls "very rare in practice" because of **failover slots** ≈ *replication slots that are synchronized to a standby, so a consumer can carry on from the same position after a failover*.[4]

Deletes and appends, not MERGE

On the Snowflake side, the apply task doesn't upsert. Slot lists three problems with the common upsert approach: the destination passes through inconsistent intermediate states, especially when a table is added; inserts get expensive because each one has to be matched against existing rows in **columnar storage** *≈ a layout that stores each column's values together, which makes scans and aggregates fast but makes finding and changing individual rows expensive*; and it's hard to combine recent changes with the target table efficiently.[4]

Instead, the change stream is turned into "perfect deletions and insertions that are applied exactly once". In Slot's words, "inserts are appended, never upserted".[4] The docs spell out the consequence: because ongoing apply is delete-then-append rather than `MERGE`, an insert-only batch skips the scan of the target table that `MERGE`-based tools pay on every run.[2]



Contrast as described in Snowflake's design post and docs. The right-hand example is a schematic, not real syntax.

Updates still cost something: an update is a delete of the old row plus an insert of the new one, and deletes have to find their rows. The gain is largest for insert-heavy tables such as events, orders and logs.

Freshness: refresh_interval and the $live view

The apply task runs on a schedule you choose with `refresh_interval`. The default is **10 minutes**; accepted values are 30 seconds, 1 minute, 10 minutes, 1 hour and 1 day, and 1 day is the maximum.[11] A shorter interval keeps targets fresher with more, smaller apply runs; a longer one is cheaper because more changes are combined per run.[5]

You don't have to choose between cheap and fresh, though, because of the `$live` view. It combines the target table with the change-log rows that haven't been applied yet, and filters and projections are pushed down into scans of both the change-log **Parquet** *≈ a compressed, column-oriented file format widely used for analytics data on object storage* files and the base table.[4] With default settings, changes are visible through `$live` within about 30 seconds, independent of `refresh_interval`.[5] So the docs suggest pairing a long interval with `$live` for low-latency reads.[5] Schema changes don't wait either: DDL, renames and snapshots apply immediately, and the apply task checks for work at least every two minutes.[2,5]

Setting one up

You create a mirror with a stored procedure in Snowflake, `SNOWFLAKE.POSTGRES.CREATE_MIRROR`. It takes a mirror name, the Postgres instance and database, a target database that **must not already exist**, and either `postgres_tables` (exactly the tables you list), `postgres_schemas` (every table in the schema, now and in future), or both. Optional arguments are `refresh_interval`, `target_table_type` (Snowflake tables by default, or `'ICEBERG'`), a `warehouse` (serverless compute by default) and an `external_volume` for writing Iceberg targets to your own storage.[12,11]

Create a mirror, then check its lag

Create (illustrative)

```
CALL SNOWFLAKE.POSTGRES.CREATE_MIRROR(
  mirror_name       => 'shop_mirror',
  postgres_instance => 'shop_pg',
  postgres_database => 'shop',
  target_database   => 'SHOP_ANALYTICS',
  postgres_schemas  => ['public'],
  refresh_interval  => '1 hour');
```

Monitor

```
CALL SNOWFLAKE.POSTGRES.DESCRIBE_MIRROR(
  'shop_mirror');
-- postgres_status:cdc_flush_lag_seconds
-- apply_lag_seconds

-- on the Postgres side:
SELECT * FROM
  snowflake_cdc.publication_health;
```

Argument names and defaults are from Snowflake's mirror procedures reference; the value syntax on the left is illustrative, so check the reference before copying. Monitoring calls are from the performance page.

Schema-level mirroring picks up new tables automatically, including their initial snapshot, while table-level mirroring is the safer choice when a schema also holds unsupported or sensitive tables.[2] Initial snapshots use parallel binary `COPY` streamed into compressed Parquet.[2] Lag is split in two: `cdc_flush_lag_seconds` (how far the Postgres side is behind; it "stays within one push interval" while the worker keeps up) plus `apply_lag_seconds` (how far the Snowflake side is behind). Add them for end-to-end lag.[5]

What your types become

Most types map directly, but the edges are worth reading before you rely on them. In the Iceberg change logs:[13]

- `numeric(p,s)` with precision and scale up to 38 maps to `decimal`. **Unbounded `numeric`, or precision above 38, becomes `double`**, so exact decimal digits may not be preserved and values beyond the `float8` range overflow to infinity. NaN in a bounded `numeric` becomes NULL.
- `text` and `varchar` values over **16 MiB are truncated**; `bytea` over **8 MiB** is truncated.
- `json` and `jsonb` are cast to `string`, and oversized values are set to **NULL**, because "truncating the serialized text would produce invalid JSON".
- Only one-dimensional arrays survive; multidimensional values become NULL. `uuid` works as a column but not inside arrays or composite types.

- Infinite and out-of-range timestamps are clamped to the nearest Iceberg boundary.

WHAT IT MEANS FOR YOU

If you already run Snowflake Postgres and copy data into Snowflake with another tool, mirroring is now the supported, in-platform way to do it, and it's worth a side-by-side test. Snowflake's sizing advice is specific: start with a STANDARD_XL instance, allow "at least one core per mirror", keep frequently changed tables spread across several mirrors, keep staging tables out, and split bulk loads into smaller transactions.[5] The `BURSTABLE` tier isn't supported at all.[12]

Budget for two kinds of cost. The apply task, named `APPLY_MIRROR_<mirror_name>`, is billed at the standard serverless task rate, and storage is billed on both sides: the `$changes` and metalog tables on the Postgres side and the targets in Snowflake.[2] The docs give the account-usage views to watch rather than prices. Remember too that mirroring runs on your database: it shares CPU and memory with your application, its work grows with write volume, and it holds each transaction's changes until commit, so big transactions use more memory and disk and show up only after they commit.[5]

If your Postgres lives somewhere else, this doesn't help you directly, because mirroring only works from Snowflake Postgres. But the design is a useful yardstick. Compare Databricks' integrated MySQL CDC pipeline, which went GA the same day: it pulls from the MySQL **binlog** *≈ MySQL's binary log of row changes, the MySQL equivalent of the WAL for replication and CDC* during each scheduled pipeline update, writes extracted changes to a staging volume, then merges them into streaming tables using primary keys. It needs classic compute, and if the binlog is purged before changes are read, the affected tables need a full refresh.[14] That's the conventional pull-and-merge shape, done well. Snowflake's bet is that pushing from inside the source and appending instead of merging removes whole classes of problems. Because the change logs are plain Iceberg tables, it also gives you a seven-day change feed you can query with SQL from both systems.[2]

💥 Watch out for

- **Tables without a primary key become insert-only.** The CDC extension sets `REPLICA IDENTITY NOTHING` on them, and Postgres then rejects `UPDATE` and `DELETE` on those tables.[12] If your application updates a keyless table, adding it to a mirror will break those writes. Add a primary key first.
- **One replication slot per mirror.** The default `max_replication_slots` is 10, and that limit is shared with any other logical replication you run.[12]
- **Unsupported objects:** views, materialized views, foreign tables, sequences, temporary and unlogged tables aren't mirrored; tables with virtual generated columns make `CREATE_MIRROR` fail; column names that are ANSI reserved words are rejected.[12]
- **Silent data changes at the edges:** unbounded `numeric` becomes floating point, and oversized JSON becomes NULL (see the type list above).[13]
- **The target database must be new.** You can't reuse one, even from a dropped mirror. External Iceberg catalogs aren't supported.[12]
- **No published benchmarks.** "Sub-minute" lag and the ~15-second push cadence are Snowflake's descriptions; benchmark on your own instance size.[2,5]

WHAT HAPPENS NEXT

The obvious next step is the reverse direction. In June, Snowflake said Snowflake-to-Postgres mirroring was planned "later this year".[3] The other thing to watch is whether the `snowflake_cdc` design, or pieces of it, end up in the open-source pg_lake project. The docs describe `snowflake_cdc` as part of Snowflake Postgres, and nothing we read says it will be open-sourced.

📖 Words from this story

Apache Iceberg: an open table format: Parquet files on object storage plus metadata tracking each table version.

Foreign data wrapper: a Postgres extension type that exposes outside data as a queryable table.

DuckDB: an embedded, column-oriented analytical database engine.

Change data capture (CDC): capturing every insert, update and delete as a stream so another system can stay in sync.

Write-ahead log (WAL): Postgres's append-only log of every change, used for recovery and replication.

Logical decoding: turning WAL records back into row-level changes for outside consumers.

Output plugin: code that formats decoded changes for a consumer, such as `pgoutput`.

Replication slot: a crash-safe bookmark that remembers a consumer's position and retains the WAL it needs.

Replica identity: what Postgres logs to identify the old row on update or delete (primary key, index, FULL or NOTHING).

Debezium: a widely used open-source CDC tool, usually feeding Kafka.

Upsert: insert-or-update by key, usually done with `MERGE`.

Publication: a Postgres object naming the tables whose changes are published.

LSN: log sequence number, a position in the WAL.

Failover slots: replication slots synchronized to a standby so CDC survives a failover.

Columnar storage: storing each column's values together; fast scans, expensive row changes.

Parquet: a compressed, column-oriented file format for analytics data.

Binlog: MySQL's binary log of row changes.

STORY 2 Postgres 19 lost half its features. AI reviewers are part of why

POSTGRESQL 19 · EISENTRAUT POST 9 OCT, LWN 13 OCT, THE REGISTER 16 OCT · GA PLANNED 19 NOV 2026 · ~13 MIN READ

Open-source databases have always had more ideas than reviewers. For decades the bottleneck was finding bugs: a subtle flaw in a big feature might surface years after release. This year that changed for PostgreSQL, and the effects are now visible. Release 19 will arrive about two months later in the year than 18 did, roughly ten major features have been pulled from it, and one quarterly update this summer fixed four times as many security bugs as the project fixed in all of 2025. This week one of the committers whose work was reverted explained why, and his explanation is a preview of what AI-assisted review is going to do to every large codebase, including the databases you run.

WHAT HAPPENED

We've covered PostgreSQL 19 twice: the Beta 4 reverts in Issue #1 and the RC 1 fixes in Issue #12. This story is about something new: an explanation, from inside the project, of why this cycle went the way it did.

On **9 October 2026**, **Peter Eisentraut**, a PostgreSQL core team member and the developer or committer of some of the affected features, published "The reverts will continue until morale improves". He counts about **8 to 10 significant features** reverted from the 19 branch since beta began, against a normal rate of about one revert per cycle.[15] He gives several overlapping causes, and two of them are new: **LLM-assisted code review**, which now finds defects much faster than before, and what he calls the **"Vulnpocalypse"**, a flood of security work that kept senior developers busy from feature freeze until the August security release.[15] His summary of the result: PostgreSQL 19 now has roughly **half the major features** he expected a few months earlier.[15]

LWN covered the post on 13 October,[16] and on 16 October The Register ran it under the headline "Postgres 19 delayed after AI reviewers find more bugs than humans can fix".[17] The schedule itself was settled a few days earlier. On 10 October, Jonathan Katz of the release team announced that RC 1 would go ahead on **15 October**, that a second release candidate would follow on **12 November**, and that general availability would move to **19 November**, a week after the November update release, because "a large number of fixes" had built up in the older branches.[18] RC 1 shipped as planned.[19]

THE BACKGROUND YOU NEED

How a Postgres release normally runs

PostgreSQL ships one **major release** *≈ a yearly feature release, such as 19, that can change on-disk formats* a year. New features land on the development branch until the **feature freeze** *≈ the date after which a release accepts only bug fixes, no new features*; for 19 that was **8 April 2026**.[20] Then come betas (Beta 1 on 4 June this year), release candidates, and usually a September or early October GA.[19] For comparison, PostgreSQL 18 had its RC on 4 September 2025 and went GA three weeks later.[20]

Between majors, the project ships **minor releases** *≈ bug- and security-fix updates to every supported version, such as 18.6, installed by swapping binaries and restarting*, normally once a quarter, plus out-of-cycle releases when

something urgent comes up.[21]

A **revert** ≈ *undoing a committed change in version control, here meaning a feature is removed from the release branch* during beta isn't a failure of the process. It's the process working. The project would rather pull a feature than ship one it can't stand behind. What's unusual about 19 is the number of reverts, and how late they came.

The features that left

The reverted list includes some of the most anticipated work in the release:[22,23]

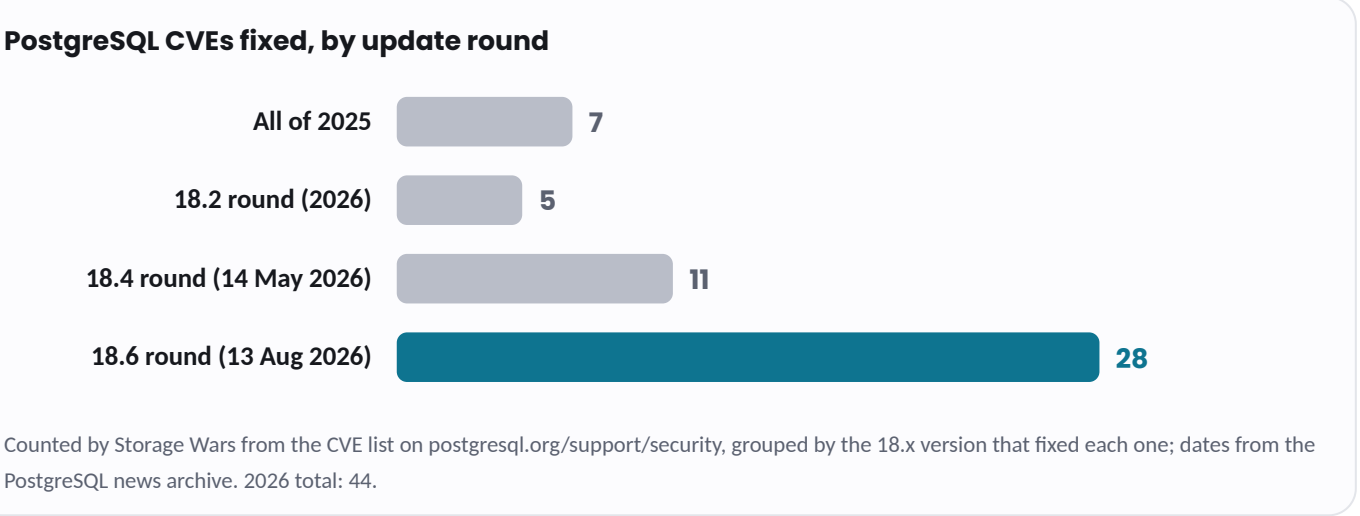
- **SQL/PGQ**, **property graph queries** ≈ *SQL standard syntax, added in SQL:2023, for querying data as nodes and edges (who follows whom, which part feeds which machine) on top of ordinary tables*, reverted on 7 September in a revert that removed 47 commits.[23] Pettus puts it at 124 files and 15,859 deleted lines.[20]
- `ALTER TABLE ... MERGE/SPLIT PARTITION`, reverted on 27 August (14 commits), citing "multiple design issues which are too late to address in this release cycle".[23]
- `UPDATE/DELETE ... FOR PORTION OF`, for **temporal tables** ≈ *tables where each row is valid for a time range, so you can change "the price from March to June" without touching other periods*.
- `GROUP BY ALL`, after review found it mishandled some `ORDER BY` entries and could give wrong results.
- **Online data checksums**, which would have let you turn page checksums on or off while the server runs.
- **Non-text output formats for** `pg_dumpall`, and **batching for foreign-key checks**.[22,24]

Most are expected to come back in PostgreSQL 20.[22,15]

HOW IT WORKS: THREE FORCES THAT COLLIDED

1. A security wave found by machines

Start with the numbers on the project's security page. All of 2025 produced **7** PostgreSQL CVEs. In 2026 so far, the 18.2 update round fixed **5**, the 18.4 round (14 May) fixed **11**, and the 18.6 round on **13 August** fixed **28**, for a total of **44**.[25,19]



Counted by Storage Wars from the CVE list on postgresql.org/support/security, grouped by the 18.x version that fixed each one; dates from the PostgreSQL news archive. 2026 total: 44.

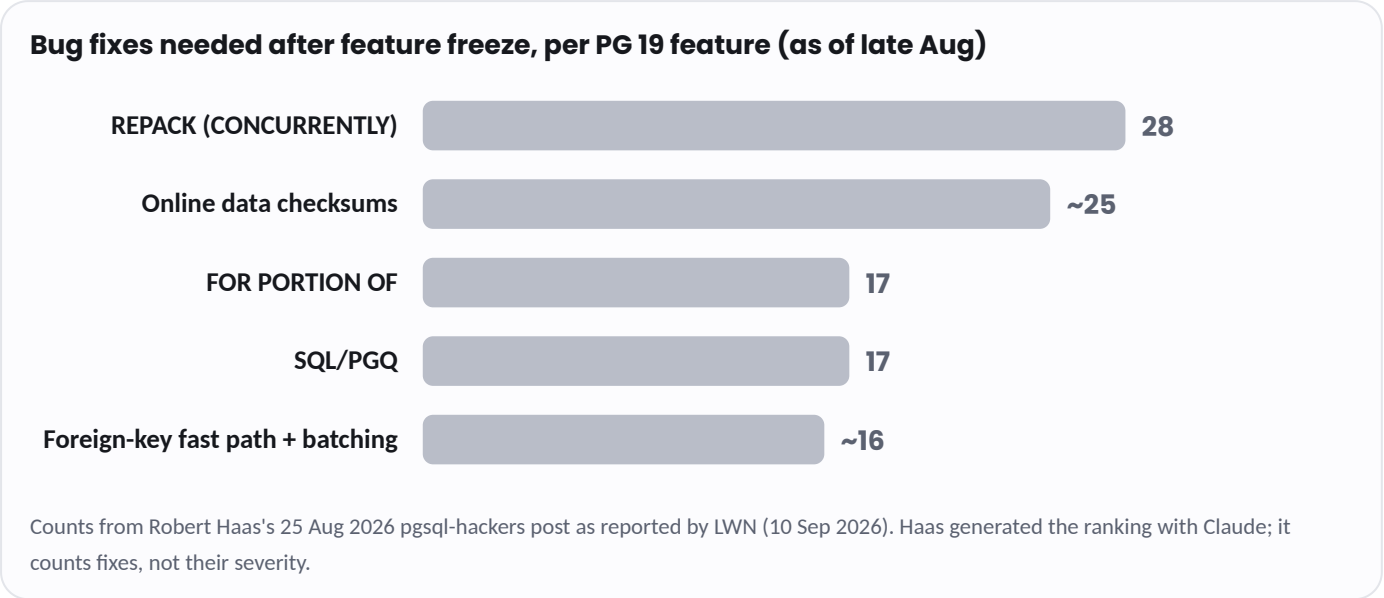
Many of the 28 August CVEs are serious: 17 have a CVSS score of 8.1 or higher, including heap buffer overflows in `to_char()`, regular expressions and `pg_stat_statements` that the advisory says can execute

arbitrary code.[26] The release notes show who found some of them. Several credit "Ben Morris (in collaboration with Claude and Anthropic Research)", and others credit "Amy Burnett (OpenAI Codex Security)".[27] One `to_char()` overflow credits eleven separate reporters.[27]

Each of those fixes had to be written, reviewed, back-patched to as many as five supported branches and released under embargo, mostly by the same senior developers who would normally spend the summer testing the next major release. That's Eisentraut's "Vulnpocalypse": from feature freeze through the August security release, the people who usually review beta features were busy with security. He believes two more months might have fixed many of the issues, but the community chose to keep the schedule.[15]

2. An AI-assisted triage of the riskiest features

On **25 August**, committer Robert Haas posted what he called a "scary patch contest" to the developers' list. He asked Claude to rank the release's patches by how many post-freeze bug fixes each had needed.[24] LWN reported the leaders:



Counts from Robert Haas's 25 Aug 2026 pgsql-hackers post as reported by LWN (10 Sep 2026). Haas generated the ranking with Claude; it counts fixes, not their severity.

The list was a conversation starter, not a verdict. What followed were human judgments, feature by feature.[24,20]

- **SQL/PGQ** had lower-severity fixes, but Tom Lane predicted that problems like constraints that could be dropped out from under a graph would need catalog changes, which can't happen until 20. "At this point I'd be willing to bet dinner that if we ship it in v19 there will be post-release bug discoveries that are unfixable until v20," he wrote.[17,20] It was reverted.
- **Foreign-key batching** was removed by its own author, Amit Langote, because buffered checks could let a transaction commit a permanent foreign-key violation without an error. The per-row fast path stayed.[20,24]
- **FOR PORTION OF** drew a report from Andres Freund of lost updates under READ COMMITTED ≈ *Postgres's default isolation level, where each statement sees data committed before it started*, and its fix count passed 30.[20,24]
- **REPACK (CONCURRENTLY)**, the top of the list, **stayed**, but with its scope cut. Pettus reports that it now rejects non-heap table access methods, user catalog tables and materialized views.[20] In Issue #12 we covered one of its last fixes before RC 1.

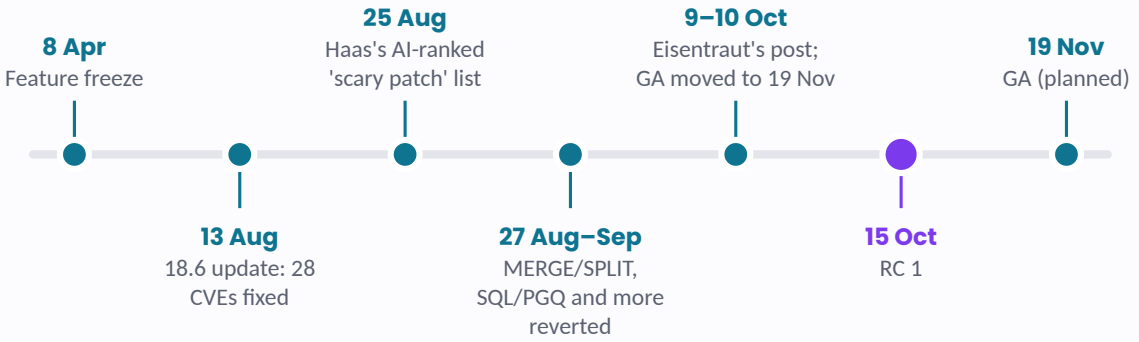
3. Forty small reports, each with a fixed cost

The third force is the one Eisentraut spends most time on. Some reverted features had serious architectural defects. Others had a long tail of edge cases, the kind of bug that in the past might have turned up years after release. "With LLM-assisted code review, these kinds of things can get found much faster," he wrote. You end up staring at a list of forty defect reports, and "even if each of them requires only a three-line fix on average, there is a fixed overhead to dealing with each of these". Each one needs reproducing, a patch, review, a test, a commit, sometimes a back-patch. Multiply by forty and the beta period runs out.[17,15]

His conclusion is that "the effective quality bar has been raised after the time the feature was written and committed".[17] The code wasn't written with AI help, and he's explicit that this is "not due to 'vibe coding'", because most of it predates that being possible.[15] It was written to the old standard of scrutiny and then reviewed to a new one.

He also names causes that have nothing to do with AI: some work may simply not have been good enough; the project may have taken on features too hard to finish in time; and Postgres's principle that "everything works with everything" (domains, composite types, partitions, views, triggers, security-definer functions all combining freely) makes features very hard to test exhaustively. He suggests LLM-assisted **fuzzing** ≈ *automatically generating huge numbers of unusual inputs to find crashes and wrong results* should become part of feature development, and he wants the project to settle how features could be committed as "experimental" so they can mature in the main tree.[15]

The PostgreSQL 19 cycle, 2026



Dates from Pettus (feature freeze), the PostgreSQL news archive, LWN, Command Prompt, Eisentraut's blog and Jonathan Katz's 10 Oct 2026 pgsql-hackers post.

WHAT IT MEANS FOR YOU

PostgreSQL 19 is smaller, and that's not bad news. What's left still includes `REPACK (CONCURRENTLY)` for reclaiming space without the long lock of `VACUUM FULL`, logical replication of sequences, `pg_plan_advice`, parallel autovacuum, `ON CONFLICT DO SELECT` and `IGNORE NULLS` in window functions.[15,22] Eisentraut says it will be "very robust, after all the additional scrutiny".[17] If you were waiting for graph queries, partition merge/split or temporal `FOR PORTION OF`, plan for PostgreSQL 20 instead.

Expect more Postgres patch releases. On 4 September, Nathan Bossart proposed releasing minor versions more often, "given the recent influx of bug reports". Three options were heading for a vote by the release

team: the second Thursday of every even month or every odd month (6 releases a year instead of 4), or an extra release halfway between each quarterly one (8 a year, roughly every 6–7 weeks). He suggested starting in 2027 as a trial. The trade-offs he named are more release work for the project and "extra restarts for users".[21] If you patch Postgres by hand, that's more maintenance windows. If you're on a managed service, watch how fast your provider follows.

Take the minor releases seriously. The August fixes weren't just crash bugs. Several of the 8.8-rated CVEs are in everyday SQL functionality, such as `to_char()`, regular expressions and the text search types, not just in obscure tools.[26] Some fixes also change behaviour: 18.6 now rejects calls from SQL to functions that take or return the internal-only type `internal`.[27] Read the release notes for anything your extensions or functions do.

PostgreSQL 14 gets its last update on 12 November, the same day as RC 2.[26] If you're still on 14, this cycle's CVE rate is a strong argument for moving now, not after the next round.

And the lesson is bigger than Postgres. If AI tools can find years' worth of latent bugs in one of the most carefully reviewed C codebases around, other databases, drivers and poolers will see the same wave. We've already covered several in this newsletter: PgBouncer and Pgpool-II in Issue #3 and pgjdbc in Issue #6.

💥 Watch out for

- **"AI caused the delay" is too simple.** Eisentraut lists LLM review as one cause among several, alongside code quality, overreach, the security workload and system complexity.[15] An LWN commenter first argued the framing overstated AI's role, then conceded it was fair.[16]
- **The ranking was a count, not a risk score.** Haas's list counted post-freeze fixes per patch; it didn't weigh severity or who could trigger a bug.[24]
- **Secondary write-ups disagree in the details.** Snowflake's 16 September blog says 53 reverts were made "since the beta began in June 2025" (the beta began in June 2026), and its claim that the lz4 TOAST default was reverted was corrected on the hackers list: the default still switches to lz4 when Postgres is built with lz4.[22,28]
- **The new release schedule isn't decided.** It's a proposal awaiting a vote, and Bossart frames it as a trial.[21]
- **The GA date can still move.** The release team says 19 November holds "unless an issue is discovered".[18]

WHAT HAPPENS NEXT

The dates to watch are **12 November** (RC 2, plus minor releases for every supported branch and PostgreSQL 14's final release) and **19 November** (planned GA).[18,26] After that, PostgreSQL 20 development starts with an unusual amount of nearly finished work queued up: SQL/PGQ, `MERGE/SPLIT PARTITION`, `GROUP BY ALL`, temporal updates and online checksums are all expected back.[22,15] Two process questions are open as well: the minor-release vote, and whether the project adopts an "experimental feature" mechanism. Snowflake's blog also notes that the project has no official policy yet on AI-assisted contributions, and expects one.[22] Eisentraut thinks the turbulence should "sort itself out for future development cycles", as code starts life reviewed to the new standard.[15]

📖 Words from this story

Major release: a yearly feature release, such as 19, that can change on-disk formats.

Feature freeze: the date after which a release accepts only fixes.

Minor releases: bug- and security-fix updates such as 18.6, installed by swapping binaries and restarting.

Revert: undoing a committed change; here, removing a feature from the release branch.

Property graph queries (SQL/PGQ): SQL:2023 syntax for querying tables as nodes and edges.

Temporal tables: tables whose rows are valid for time ranges, updated per period.

READ COMMITTED: Postgres's default isolation level; each statement sees data committed before it began.

Fuzzing: generating huge numbers of unusual inputs automatically to find crashes and wrong results.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **TiDB Cloud adds a PostgreSQL-compatible option:** PingCAP opened a public preview of PostgreSQL-compatible instances on TiDB Cloud Starter, alongside a BYOC preview and a Premium tier due in November, and said it will open-source the core of its TiDB X architecture by the end of 2026 (15 Oct).[29]
- **Databricks' integrated MySQL CDC pipeline is GA:** one Lakeflow pipeline now both reads the MySQL binlog and applies changes, with no separate ingestion gateway (15 Oct).[14,30]
- **PlanetScale is now available on Azure,** per its changelog, the same week it added huge-pages support for its Postgres and Neki databases (13–15 Oct).[31]
- **ClickHouse Cloud executable UDFs are GA,** adding a native runtime for statically linked Rust, Go, C++ and Bun-compiled JavaScript binaries, outbound network access and a `deterministic` flag for the query cache (5 Oct).[32]
- **Kafka 4.4.0 still isn't out,** and its release candidates already break Aiven's tiered-storage plugin, because a utility class moved to an internal package; a fix PR was opened on 10 October.[33,34]
- **Apache Auron became a top-level project:** the native, vectorized accelerator for Spark-style engines says it runs about twice as fast as Spark 3.5 on half the resources (project claim), alongside new TLPs Reqsign and ResilientDB (14 Oct).[35]
- **ClickHouse's CostBench series added BigQuery, Databricks and Redshift Serverless,** with ClickHouse claiming hundreds of times better performance per dollar on its own real-time benchmark (vendor benchmark, October).[36]

1. **Snowflake server release notes and feature updates**, entry "Data mirroring for Snowflake Postgres (General availability)", 15 Oct 2026.
2. **Data mirroring**, Snowflake documentation, viewed 17 Oct 2026.

3. **Snowflake Postgres Unifies Your Apps, Analytics and AI**, Craig Kerstiens, Claire Peracchio and Christopher Heim, Snowflake blog, 16 Jun 2026.
4. **How we pushed CDC into Postgres, and turned replication into clockwork**, Marco Slot, Snowflake engineering blog, 23 Jul 2026.
5. **Postgres-to-Snowflake mirroring performance considerations**, Snowflake documentation, viewed 17 Oct 2026.
6. **Snowflake's PostgreSQL push**, The Register, 5 Nov 2025.
7. **Snowflake Postgres (General availability)**, Snowflake release notes, 24 Feb 2026.
8. **pg_lake README**, Snowflake-Labs on GitHub, viewed 17 Oct 2026.
9. **Logical Decoding Concepts**, PostgreSQL documentation (the same text appears in current versions).
10. **Postgres REPLICA IDENTITY**, Illuminated Computing, Oct 2024.
11. **Mirror procedures reference**, Snowflake documentation, viewed 17 Oct 2026.
12. **Create a mirror**, Snowflake documentation, viewed 17 Oct 2026.
13. **Type mapping reference**, Snowflake documentation, viewed 17 Oct 2026.
14. **Integrated CDC pipeline for MySQL**, Databricks documentation, updated 15 Oct 2026.
15. **The reverts will continue until morale improves**, Peter Eisentraut, 9 Oct 2026.
16. **Eisentraut: The reverts will continue until morale improves**, LWN.net, 13 Oct 2026, and its comments.
17. **Postgres 19 delayed after AI reviewers find more bugs than humans can fix**, Lindsay Clark, The Register, 16 Oct 2026.
18. **Updated PostgreSQL RC / GA dates**, Jonathan S. Katz, pgsql-hackers, 10 Oct 2026.
19. **PostgreSQL news archive**, postgresql.org, viewed 17 Oct 2026 (RC 1 on 15 Oct, 18.6 on 13 Aug, 18.4 on 14 May, Beta 1 on 4 Jun 2026).
20. **19th Nervous Breakdown**, Christophe Pettus, The Build, 15 Sep 2026 (updated 24 Sep).
21. **proposal for a new minor release schedule**, Nathan Bossart, pgsql-hackers, 4 Sep 2026.
22. **What Is Happening with PostgreSQL 19?**, Elizabeth Garrett Christensen, Snowflake engineering blog, 16 Sep 2026.
23. **Two features just left PostgreSQL 19**, Joshua D. Drake, Command Prompt, 8 Sep 2026.
24. **PostgreSQL 19's "scary patch contest"**, Joe Brockmeier, LWN.net, 10 Sep 2026.
25. **Security Information**, PostgreSQL, viewed 17 Oct 2026.
26. **PostgreSQL 18.6, 17.11, 16.15, 15.19, 14.24 and 19 Beta 3 Released!**, PostgreSQL, 13 Aug 2026.
27. **Release 18.6**, PostgreSQL 18 documentation, appendix E.1.
28. **Re: PG 19 status blog report**, Bruce Momjian quoting Michael Paquier, pgsql-hackers, 19 Sep 2026.
29. **What's New in TiDB Cloud: Premium, BYOC, Postgres & More**, Brian Foster, PingCAP blog, 15 Oct 2026.
30. **Databricks platform release notes, October 2026**, entry dated 15 Oct 2026.
31. **PlanetScale changelog: PlanetScale is now available on Azure**, 13 Oct 2026.
32. **Executable UDFs are generally available on ClickHouse Cloud**, ClickHouse blog, 5 Oct 2026.
33. **Kafka 4.4.0 compatibility is broken due to missing classes (issue #842)**, Aiven-Open/tiered-storage-for-apache-kafka, 9 Oct 2026.
34. **Apache Kafka downloads**, viewed 17 Oct 2026 (newest feature release listed: 4.3.1).
35. **Apache Names Auron, Reqsign and ResilientDB Top-Level Projects**, BigDATAwire, 14 Oct 2026.
36. **ClickHouse October 2026 newsletter**, ClickHouse, 15 Oct 2026.